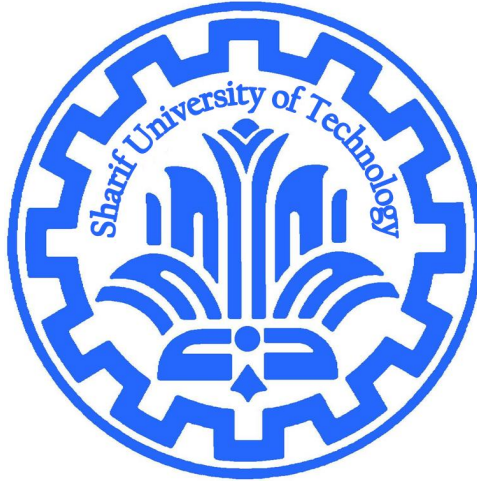


Data Networks



BitTorrent, A peer to peer file sharing system

Department of Electrical Engineering
Sharif University of Technology

Instructor: Dr. Mohammad Reza Pakravan

Submitted by:

Amirali Yazdiani - 402102765

Date:
Aug 2026

Contents

1	Torrent File Generation	2
1.1	Implementation	2
1.2	Why Bencoded Format?	3
2	Tracker Web Service Implementation	3
2.1	Concurrency and Architecture	3
2.2	State Management (Swarm Database)	3
2.3	Request Parsing and Event Handling	4
2.4	Bencoded Response and Logging	4
3	Peer Client Implementation	4
3.1	Client Initialization and Port Binding	4
3.2	Torrent Thread Pool and Network Interaction	4
3.3	End to End Peer Communication (Ping Protocol)	5
3.4	Graceful Shutdown	5
4	System Logging and Event Tracking	5
4.1	Logger Architecture and Isolation	5
4.2	Formatting and Metadata Management	6
A	Python Source Codes	7
A.1	Torrent Generator Script	7
A.2	Tracker Web Service Script	9
A.3	Peer Client Script	12
A.4	Logging Utility Script	17

1 Torrent File Generation

1.1 Implementation

To meet the project requirement of having human readable and modifiable Metainfo (.torrent) files, the initial configurations were defined using JSON format. This approach simplifies testing and parameter modification by avoiding direct hex or binary editing.

Two distinct JSON templates were developed to handle both multi and single file torrent architectures, as illustrated bellow.

Listing 1: Multi File Torrent

```
{
  "announce": "http://127.0.0.1:8000/announce",
  "info": {
    "name": "Transferring_These_Files",
    "piece length": 262144,
    "pieces": "",
    "files": [
      {
        "length": 1116449,
        "path": ["sample1.png"]
      },
      {
        "length": 1455741,
        "path": ["sample2.png"]
      }
    ]
  }
}
```

Listing 2: Single File Torrent

```
{
  "announce": "http://127.0.0.1:8000/announce",
  "info": {
    "name": "Transferring_These_Files/sample3.png",
    "length": 1455741,
    "piece length": 262144,
    "pieces": ""
  }
}
```

To convert these JSON structures into standard bencoded torrent files, a Python script ([torrent_generator.py](#)) was implemented. The script executes the following data pipeline:

- Parses the JSON configuration to extract the file structures, target paths, and the specified `piece length`.
- Maps the parsed paths to the actual physical files within the local directory.
- Iterates over the files, reading them in discrete byte chunks according to the `piece length`.
- Computes the SHA-1 hash for each chunk and concatenates them to form the final `pieces` byte string.

- Injects this hash string back into the dictionary, encodes the entire structure using the Bencode standard, and writes the output to a text file.

Executing this script on the provided JSON files successfully generates `torrent.txt`. These generated bencoded files serve as the baseline Metainfo fed into the tracker and peer instances for the remainder of the simulation. All related implementation files and outputs for this phase are contained within the `torrent generator` directory.

1.2 Why Bencoded Format?

The decision to serialize the Metainfo file using the Bencode format, rather than retaining it as standard JSON, is driven by the strict requirements of the BitTorrent protocol.

While JSON is highly suitable for human readable configurations, it lacks native support for raw binary data, such as the concatenated SHA-1 byte strings residing in the `pieces` field. Bencoding resolves this by seamlessly handling both ASCII text and arbitrary byte arrays through explicit length prefixing.

Furthermore, Bencoding enforces strict deterministic serialization by lexicographically sorting dictionary keys. This determinism is critical from a network engineering perspective: the `info_hash` parameter, which peers use to identify the torrent to the tracker, is computed by hashing the exact byte sequence of the bencoded `info` dictionary. Any variation in key ordering or whitespace, which JSON inherently allows would result in a completely different hash, fundamentally breaking the peer discovery mechanism.

2 Tracker Web Service Implementation

The tracker serves as the centralized coordinator for the BitTorrent network. It does not participate in the actual file transfer but acts as a discovery service, maintaining the state of the swarm and enabling peers to locate each other. The complete implementation of this service is provided in Appendix ([Tracker.py](#)).

From an engineering perspective, the tracker was designed with three primary considerations: concurrency, efficient state management, and strict protocol adherence.

2.1 Concurrency and Architecture

In a practical P2P environment, the tracker must handle simultaneous `GET` requests from numerous peers without blocking. To achieve this, the service is built upon Python's `ThreadingHTTPServer`. This architecture assigns a separate thread to each incoming HTTP request, ensuring that the tracker remains highly responsive even during swarm initialization bursts.

2.2 State Management (Swarm Database)

The state of the network is maintained in memory using a dictionary data structure (`swarm_db`). The database uses the `info_hash` of the torrents as keys, mapping each hash to a list of active peer dictionaries (containing `peer_id`, `ip`, and `port`).

While a persistent database (like SQL) could be used for a production scale tracker, an in memory dictionary provides the necessary $O(1)$ lookup time and is highly optimal for the scope of this simulation.

2.3 Request Parsing and Event Handling

When a peer connects to the `/announce` endpoint, the `TrackerHandler` parses the URL-encoded query parameters. The logic explicitly handles three critical events defined by the BitTorrent protocol:

- **started:** Registers a new peer in the `swarm_db` or updates its port if it already exists.
- **completed:** Flags the peer as a "seeder" (by logically treating `left = 0`). This is vital for tracking the ratio of seeders to leechers.
- **stopped:** Facilitates a graceful exit. The tracker actively removes the peer from the swarm list, preventing the distribution of dead IP addresses to other peers and thereby maintaining swarm health.

2.4 Bencoded Response and Logging

Once the swarm state is updated, the tracker dynamically generates a response containing the tracker `interval`, the count of complete/incomplete peers, and the requested peer list.

Crucially, this dictionary is serialized into a Bencode format before transmission via the HTTP 200 OK response, ensuring compatibility with standard BitTorrent clients. Throughout this lifecycle, a custom `BitTorrentLogger` synchronously records all network events, parameter anomalies, and HTTP responses to a local log file for offline debugging.

3 Peer Client Implementation

The peer client is the core entity in the BitTorrent ecosystem, responsible for interacting with the tracker and establishing direct communication with other peers.

The complete implementation for this module is provided in Appendix ([peer.py](#)). This design leverages threading to achieve concurrent execution and robust network management.

3.1 Client Initialization and Port Binding

Upon execution, the peer randomly generates a 20 byte `peer_id` (starting with `-AA0001-`) to uniquely identify itself across the swarm.

A dedicated listener thread is spawned immediately to handle incoming direct TCP connections from other peers. According to the BitTorrent protocol specification, the client attempts to bind its listening socket to port 6881. If this port is occupied, the application iteratively attempts sequential ports up to 6889, terminating gracefully if all predefined ports are exhausted.

3.2 Torrent Thread Pool and Network Interaction

A critical requirement of this project was the implementation of a Torrent Thread Pool, enabling a single peer instance to process multiple `.torrent` files simultaneously. For each torrent file:

1. **Parsing and Hashing:** The thread decodes the bencoded Metainfo, extracts the `announce` URL, and computes the strictly deterministic SHA-1 `info_hash` from the raw bytes of the `info` dictionary.
2. **Tracker Registration:** An initial HTTP `GET` request is dispatched to the tracker with the `started` event, passing required parameters including the `info_hash`, `peer_id`, and the successfully bound listening port.
3. **Periodic Announcing:** After the initial response, the thread enters a loop governed by the tracker's `interval` parameter, repeatedly querying the tracker for swarm state updates to discover newly joined peers.

3.3 End to End Peer Communication (Ping Protocol)

To validate the peer discovery mechanism and ensure end to end connectivity, a lightweight ping protocol was implemented.

When a peer receives an IP and port pair from the tracker, it cross references its local `known_peers` set to prevent redundant operations. If the peer is novel, an asynchronous socket connection is initiated to the target address, transmitting a binary `PING` message. The receiving peer's listener thread actively intercepts this packet and returns a `PONG` acknowledgment.

The originating thread then logs the successful Round Trip Time (RTT), confirming bidirectional functionality at the application layer independent of the tracker.

3.4 Graceful Shutdown

Handling network disconnections gracefully is paramount to preventing zombie states within the tracker's database. By capturing `KeyboardInterrupt` signals, the client enforces a strict exit protocol. It iterates through all active torrents and dispatches a final HTTP `GET` request with the `event=stopped` parameter. This actively deregisters the client, maintaining the integrity and accuracy of the tracker's peer lists.

4 System Logging and Event Tracking

Debugging distributed systems, particularly asynchronous peer to peer protocols, requires a robust and structured logging mechanism. Standard console outputs are insufficient and often misleading when multiple threads and distinct network entities (the tracker and various peers) operate concurrently.

To address this, a centralized and modular logging utility, `logger.py`, was developed. The complete implementation is available in Appendix ([logger.py](#)).

4.1 Logger Architecture and Isolation

The `BitTorrentLogger` class is designed to enforce a strict separation of concerns. Upon instantiation, it categorizes logs based on the `component_type` parameter (Tracker or Peer) and dynamically generates a structured directory tree under the `logs/` directory.

This architectural choice ensures physical file isolation, preventing log contamination during local swarm simulations where multiple peer instances run simultaneously on the same machine.

4.2 Formatting and Metadata Management

To strictly comply with the project specifications, the logger formats the output file into distinct Header and Body sections.

- **Header:** Stores critical metadata including the exact filename, the initial creation timestamp, and a dynamically updated Last Modified timestamp.
- **Body:** Utilizes a tabulated architecture for the actual event logs. It categorizes entries by **Event Type** (`SERVER_START`, `PING_SUCCESS`, `ERROR`), **Date/Time**, and provides a detailed **Description** of the network transaction.

A notable engineering detail in this implementation is the `_update_last_modified()` method. Rather than merely appending data to the end of the file, this method actively parses the file to rewrite the header's modification timestamp upon every new `log_event()` method call. This strict temporal tracking ensures the log file's metadata perfectly reflects the boundaries of the network simulation.

A Python Source Codes

A.1 Torrent Generator Script

This section contains the implementation of the script used to generate bencoded torrent files from JSON configurations.

Code Listing 1: torrent_generator.py

```
1 import json
2 import bencode
3 import os
4 import hashlib
5
6 ### Creating Hash
7 def generate_real_pieces(file_path, piece_length):
8     pieces = b""
9     if not os.path.exists(file_path):
10         print(f"[-] Warning: File '{file_path}' not found. Generating
11         dummy pieces.")
12         return b'\x00' * 20
13
14     with open(file_path, 'rb') as f:
15         while True:
16             chunk = f.read(piece_length)
17             if not chunk:
18                 break
19             pieces += hashlib.sha1(chunk).digest()
20
21     return pieces
22
23 def create_torrent_from_json(json_path, output_torrent_path):
24     print(f"[*] Reading human-readable config from: {json_path}")
25
26     with open(json_path, 'r', encoding='utf-8') as f:
27         torrent_dict = json.load(f)
28
29     ### If Single File
30     if 'info' in torrent_dict and 'name' in torrent_dict['info'] and '
31     files' not in torrent_dict['info']:
32         file_name = torrent_dict['info']['name']
33         piece_length = torrent_dict['info']['piece length']
34
35         ### Build Hash
36         real_pieces = generate_real_pieces(file_name, piece_length)
37         torrent_dict['info']['pieces'] = real_pieces
38
39     ### If Multiple Files
40     elif 'info' in torrent_dict and 'files' in torrent_dict['info']:
41         piece_length = torrent_dict['info']['piece length']
42         root_dir = torrent_dict['info']['name']
43         all_pieces = b""
44
45         for file_info in torrent_dict['info']['files']:
46             full_file_path = os.path.join(root_dir, *file_info['path'])
47
48             all_pieces += generate_real_pieces(full_file_path,
49             piece_length)
```



```
48     torrent_dict['info']['pieces'] = all_pieces
49
50     bencoded_data = bencode.encode(torrent_dict)
51
52     with open(output_torrent_path, 'wb') as f:
53         f.write(bencoded_data)
54
55     print(f"[+] Successfully generated: {output_torrent_path}\n")
56
57 if __name__ == "__main__":
58
59     ### Reading the first Torrent
60     json_file_1 = "torrent1.json"
61     output_file_1 = "my_real_torrent_1.txt"
62
63     ### Creating bencode from simple text
64     if os.path.exists(json_file_1):
65         create_torrent_from_json(json_file_1, output_file_1)
66
67     ### Reading the second Torrent
68     json_file_2 = "torrent2.json"
69     output_file_2 = "my_real_torrent_2.txt"
70
71     ### Creating bencode from simple text
72     if os.path.exists(json_file_2):
73         create_torrent_from_json(json_file_2, output_file_2)
```

A.2 Tracker Web Service Script

This section contains the implementation of the multi-threaded HTTP tracker.

Code Listing 2: tracker.py

```

1 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
2 import urllib.parse
3 import bencode
4 from logger import BitTorrentLogger
5
6 swarm_db = {}
7 tracker_logger = BitTorrentLogger("tracker_log.txt", "Tracker")
8
9 class TrackerHandler(BaseHTTPRequestHandler):
10     def do_GET(self):
11         parsed_url = urllib.parse.urlparse(self.path)
12         params = urllib.parse.parse_qs(parsed_url.query)
13
14         if parsed_url.path == '/announce':
15             info_hash = params.get('info_hash', [None])[0]
16             peer_id = params.get('peer_id', [None])[0]
17             port = params.get('port', [None])[0]
18
19             event = params.get('event', [None])[0]
20             left = params.get('left', ['-1'])[0]
21
22             try:
23                 left = int(left)
24             except ValueError:
25                 left = -1
26
27             if not info_hash or not peer_id or not port:
28                 tracker_logger.log_event("ERROR", "Missing required
parameters from client")
29                 self.send_error_response("Missing required parameters")
30                 return
31
32             client_ip = self.client_address[0]
33             tracker_logger.log_event("REQUEST_RECV", f"Received {event
or 'regular'} request from {peer_id} at {client_ip}:{port}")
34
35             if info_hash not in swarm_db:
36                 swarm_db[info_hash] = []
37
38             if event == 'stopped':
39                 swarm_db[info_hash] = [p for p in swarm_db[info_hash]
if p['peer_id'] != peer_id]
40
41                 print(f"[-] Removed Peer (stopped): {peer_id} from {
client_ip}:{port}")
42                 tracker_logger.log_event("PEER_REMOVED", f"Peer {
peer_id} gracefully stopped and removed")
43
44                 self.send_success_response(info_hash)
45                 return
46
47             elif event == 'completed':
48                 print(f"[*] Peer {peer_id} completed the download! Now

```

```
a Seeder.")
49     tracker_logger.log_event("PEER_COMPLETED", f"Peer {
peer_id} at {client_ip}:{port} is now a seeder")
50     left = 0
51
52     elif event == "started":
53         new_peer = {
54             'peer_id': peer_id,
55             'ip': client_ip,
56             'port': int(port)
57         }
58
59         peer_exists = False
60         for p in swarm_db[info_hash]:
61             if p['peer_id'] == peer_id:
62                 peer_exists = True
63                 p['port'] = int(port)
64                 break
65
66         if not peer_exists:
67             swarm_db[info_hash].append(new_peer)
68
69         print(f"[+] Added/Updated Peer: {peer_id} at {client_ip}
:{port}")
70         tracker_logger.log_event("PEER_ADDED", f"Added/Updated
Peer: {peer_id} at {client_ip}:{port}")
71
72         self.send_success_response(info_hash, requested_by=peer_id)
73
74     else:
75         tracker_logger.log_event("WARNING", f"Invalid path
requested: {self.path}")
76
77         self.send_response(404)
78         self.end_headers()
79
80     def send_success_response(self, info_hash, requested_by=None):
81         complete_count = sum(1 for p in swarm_db[info_hash] if p.get('
left', -1) == 0)
82         incomplete_count = len(swarm_db[info_hash]) - complete_count
83
84         peers_to_send = [
85             {'peer_id': p['peer_id'],
86              'ip': p['ip'],
87              'port': p['port']}
88             for p in swarm_db[info_hash]
89         ]
90
91         response_dict = {
92             'interval': 30,
93             'complete': complete_count,
94             'incomplete': incomplete_count,
95             'peers': peers_to_send
96         }
97
98         bencoded_response = bencode.encode(response_dict)
99
100        self.send_response(200)
```

```
101     self.send_header('Content-type', 'text/plain')
102     self.end_headers()
103     self.wfile.write(bencoded_response)
104
105     if requested_by:
106         tracker_logger.log_event("RESPONSE_SENT", f"Sent peer list
107         ({len(peers_to_send)} peers) to {requested_by}")
108
109     def send_error_response(self, reason):
110         response = bencode.encode({'failure reason': reason})
111         self.send_response(200)
112         self.send_header('Content-type', 'text/plain')
113         self.end_headers()
114         self.wfile.write(response)
115
116 def start_tracker(port=8000):
117     server_address = ('', port)
118     tracker_server = ThreadingHTTPServer(server_address, TrackerHandler
119     )
120     print(f"[*] Tracker is up and running on port {port}...")
121     tracker_logger.log_event("SERVER_START", f"Tracker started on port
122     {port}")
123
124     try:
125         tracker_server.serve_forever()
126     except KeyboardInterrupt:
127         print("\n[*] Shutting down the tracker.")
128         tracker_logger.log_event("SERVER_STOP", "Tracker shut down by
129         user")
130         tracker_server.server_close()
131
132 if __name__ == '__main__':
133     start_tracker()
```

A.3 Peer Client Script

This section contains the source code for the multi-threaded peer client.

Code Listing 3: peer.py

```
1 import bencode
2 import hashlib
3 import urllib.parse
4 import urllib.request
5 import random
6 import string
7 import threading
8 import time
9 import socket
10 from logger import BitTorrentLogger
11
12 def generate_peer_id():
13     prefix = '-AA0001-'
14     random_chars = ''.join(random.choices(string.ascii_letters + string
15     .digits, k=12))
16     return prefix + random_chars
17
18 MY_PEER_ID = generate_peer_id()
19 ACTIVE_TORRENTS = []
20
21 peer_logger = BitTorrentLogger(f"peer_{MY_PEER_ID}.log", "peers")
22 PEER_PORT = 6881
23
24 def start_peer_listener():
25     global PEER_PORT
26
27     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28
29     port = PEER_PORT
30     flag = True
31     while port <= 6889:
32         try:
33             server_socket.bind(('127.0.0.1', port))
34             print(f"[+] Listener successfully started on port {port}")
35             peer_logger.log_event("LISTENER_START", f"Peer listening
36             for direct connections on port {port}")
37             PEER_PORT = port
38             flag = False
39             break
40         except OSError:
41             print(f"[-] Port {port} is busy, trying next...")
42             peer_logger.log_event("PORT_BUSY", f"Port {port} in use.
43             Trying next...")
44             port += 1
45
46     if (flag):
47         print("[-] All Ports are in use!")
48         peer_logger.log_event("ERROR", "Failed to start listener. All
49         ports in use.")
50         return
51
52     server_socket.listen(5)
```

```
50 while True:
51     try:
52         client_socket, addr = server_socket.accept()
53         data = client_socket.recv(1024).decode('utf-8')
54
55         if data == "PING":
56             peer_logger.log_event("PING_RECV", f"Received PING from {addr}")
57             client_socket.send("PONG".encode('utf-8'))
58
59             client_socket.close()
60     except Exception as e:
61         peer_logger.log_event("ERROR", f"Listener error: {e}")
62
63 def ping_other_peer(ip, port, torrent_name ):
64     try:
65         peer_logger.log_event("PING_SEND", f"[{torrent_name}] Attempting to ping peer at {ip}:{port}")
66
67         server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
68         server_socket.settimeout(2)
69         server_socket.connect((ip, port))
70
71         start_time = time.time()
72
73         server_socket.send("PING".encode('utf-8'))
74
75         response = server_socket.recv(1024).decode('utf-8')
76         if response == "PONG":
77
78             end_time = time.time()
79             rtt_ms = (end_time - start_time) * 1000
80
81             print(f"      [+] DIRECT CONNECTION SUCCESS: Received PONG from {ip}:{port} with ping {rtt_ms:.2f}")
82             peer_logger.log_event("PING_SUCCESS", f"[{torrent_name}] Received PONG from {ip}:{port} | RTT: {rtt_ms:.2f} ms")
83
84             server_socket.close()
85     except Exception as e:
86         peer_logger.log_event("PING_FAIL", f"[{torrent_name}] Failed to ping {ip}:{port} - {e}")
87
88 def process_torrent(torrent_path):
89     peer_id = MY_PEER_ID
90     print(f"\n[*] [Thread {threading.current_thread().name}] Parsing: {torrent_path} with ID {peer_id}")
91
92     try:
93         with open(torrent_path, 'rb') as f:
94             torrent_data = bencode.decode(f.read())
95
96             announce_url = torrent_data['announce']
97             info_dict = torrent_data['info']
98             bencoded_info = bencode.encode(info_dict)
99             info_hash = hashlib.sha1(bencoded_info).digest()
100             torrent_name = info_dict['name']
```

```
101     total_length = 0
102     if 'files' in info_dict:
103         for f in info_dict['files']:
104             total_length += f['length']
105     else:
106         total_length = info_dict['length']
107
108     global ACTIVE_TORRENTS
109     ACTIVE_TORRENTS.append({
110         'announce_url': announce_url,
111         'info_hash': info_hash,
112         'left': total_length
113     })
114
115     known_peers = set()
116
117     params = {
118         'info_hash': info_hash,
119         'peer_id': peer_id,
120         'port': PEER_PORT,
121         'uploaded': 0,
122         'downloaded': 0,
123         'left': total_length,
124         'event': 'started'
125     }
126
127     url_params = urllib.parse.urlencode(params)
128     request_url = f"{announce_url}?{url_params}"
129
130     response = urllib.request.urlopen(request_url)
131     tracker_response = bencode.decode(response.read())
132
133     print(f"\n[+] [Thread {threading.current_thread().name}]
134 Received peers for {info_dict['name']}")
135     peers_list = tracker_response.get('peers', [])
136
137     if peers_list:
138         for p in peers_list:
139             target_ip = p['ip']
140             target_port = p['port']
141
142             if target_port == PEER_PORT:
143                 continue
144
145             known_peers.add((target_ip, target_port))
146             ping_other_peer(target_ip, target_port, torrent_name)
147     else:
148         print("        [-] No other peers in the swarm yet.")
149
150     interval = tracker_response.get('interval', 30)
151
152     while True:
153         time.sleep(interval)
154         peer_logger.log_event("ANNOUNCE", f"[{info_dict['name']}]
155 Sending periodic update to tracker...")
156
157         update_params = {
```

```
157         'info_hash': info_hash,
158         'peer_id': peer_id,
159         'port': PEER_PORT,
160         'uploaded': 0,
161         'downloaded': 0,
162         'left': total_length
163     }
164
165     update_url = f"{announce_url}?{urllib.parse.urlencode(
166 update_params)}"
167     try:
168         update_response = urllib.request.urlopen(update_url)
169         update_tracker_data = bencode.decode(update_response.
170 read())
171
172         new_peers_list = update_tracker_data.get('peers', [])
173         for p in new_peers_list:
174             target_ip = p['ip']
175             target_port = p['port']
176
177             if target_port == PEER_PORT:
178                 continue
179
180             if (target_ip, target_port) not in known_peers:
181                 print(f"      [+] NEW PEER DISCOVERED for {
182 torrent_name}: {target_ip}:{target_port}")
183                 peer_logger.log_event("NEW_PEER", f"[{
184 torrent_name}] Discovered new peer {target_ip}:{target_port} from
185 tracker update")
186
187                 known_peers.add((target_ip, target_port))
188                 ping_other_peer(target_ip, target_port,
189 torrent_name)
190
191             except Exception as e:
192                 peer_logger.log_event("ERROR", f"[{torrent_name}]
193 Periodic announce failed: {e}")
194
195             except Exception as e:
196                 print(f"[-] [Thread {threading.current_thread().name}] Error: {
197 e}")
198                 peer_logger.log_event("ERROR", f"Torrent process error: {e}")
199
200 def start_client():
201     listener_thread = threading.Thread(target=start_peer_listener,
202 daemon=True)
203     listener_thread.start()
204
205     time.sleep(1)
206
207     torrent_files = [
208         r"Torrent_Handler\my_real_torrent_1.txt",
209         r"Torrent_Handler\my_real_torrent_2.txt"
210     ]
211
212     threads = []
213
214     for i, file_path in enumerate(torrent_files):
```



```
206     t = threading.Thread(target=process_torrent, args=(file_path,),
207     name=f"Torrent-{i+1}", daemon=True)
208     threads.append(t)
209     t.start()
210     time.sleep(1)
211
212 for t in threads:
213     t.join()
214
215 print("\n[*] All torrents processed successfully. Keeping listener
216 alive...")
217
218 try:
219     while True:
220         time.sleep(1)
221 except KeyboardInterrupt:
222     print("\n[*] Shutting down peer. Notifying tracker...")
223
224 for torrent in ACTIVE_TORRENTS:
225     stop_params = {
226         'info_hash': torrent['info_hash'],
227         'peer_id': MY_PEER_ID,
228         'port': PEER_PORT,
229         'uploaded': 0,
230         'downloaded': 0,
231         'left': torrent['left'],
232         'event': 'stopped'
233     }
234     stop_url = f"{torrent['announce_url']}?{urllib.parse.
235 urlencode(stop_params)}"
236     try:
237         urllib.request.urlopen(stop_url)
238     except Exception as e:
239         pass
240
241     print("[+] Successfully unregistered from tracker. Goodbye!")
242
243 if __name__ == '__main__':
244     start_client()
```

A.4 Logging Utility Script

This section contains the implementation of the custom logging module utilized by both the tracker and the peer clients[cite: 9].

Code Listing 4: logger.py

```

1 import os
2 import datetime
3
4 class BitTorrentLogger:
5     def __init__(self, log_filename, component_type):
6         self.log_filename = log_filename
7         self.component_type = component_type
8
9         self.log_dir = os.path.join("logs", self.component_type)
10
11         os.makedirs(self.log_dir, exist_ok=True)
12
13         self.full_path = os.path.join(self.log_dir, self.log_filename)
14
15         self.initialize_log_file()
16
17     def get_current_time(self):
18         return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
19
20     def initialize_log_file(self):
21         if not os.path.exists(self.full_path):
22             creation_time = self.get_current_time()
23             with open(self.full_path, 'w', encoding='utf-8') as f:
24                 f.write("="*50 + "\n")
25                 f.write("HEADER\n")
26                 f.write(f"File name: {self.log_filename}\n")
27                 f.write(f"Creation Date/Time: {creation_time}\n")
28                 f.write(f>Last Modified Date/Time: {creation_time}\n")
29                 f.write("="*50 + "\n")
30                 f.write("BODY\n")
31                 f.write(f"{'Event Type':<15} | {'Date/Time':<20} | {'
Description'}\n")
32                 f.write("-" * 50 + "\n")
33
34     def log_event(self, event_type, description):
35         current_time = self.get_current_time()
36
37         with open(self.full_path, 'a', encoding='utf-8') as f:
38             f.write(f"{'event_type':<15} | {'current_time':<20} | {
description}\n")
39
40         self._update_last_modified(current_time)
41
42     def _update_last_modified(self, current_time):
43         with open(self.full_path, 'r', encoding='utf-8') as f:
44             lines = f.readlines()
45
46         with open(self.full_path, 'w', encoding='utf-8') as f:
47             for line in lines:
48                 if line.startswith("Last Modified Date/Time:"):
49                     f.write(f>Last Modified Date/Time: {current_time}\n
")

```

```
50         else:
51             f.write(line)
```